

SIMULACIÓN DE RECORRIDO BFS Y DFS EN GRAFOS CON ANIMACIÓN

Para empezar utilice `!pip install networkx` una librería que nos sirve para trabajar con grafos, además importe otras librerías que me ayudaron en el desarrollo del código, `NetworkX` se utiliza para crear y trabajar con el grafo, `Matplotlib` para poder dibujarlo en pantalla, `FuncAnimation` para generar la animación del recorrido de los nodos, `HTML` para que esa animación se pueda mostrar correctamente en Google Colab y de que se usa para manejar la cola que necesita el algoritmo BFS para recorrer el grafo por niveles.

Decidí utilizar 8 nodos porque cuando probé el ejercicio con solo 6 nodos, el recorrido de BFS y DFS me daba prácticamente el mismo resultado, y no se podía notar bien la diferencia entre ambos algoritmos. Con más nodos el recorrido se vuelve más claro.

A, B, C, D, E, F, G, H

A → B, C

B → D, E

C → F, G

D → H

Utilicé `DiGraph` porque el grafo es dirigido, o sea, las conexiones tienen una dirección específica.

Luego utilicé un ciclo `for` para agregar cada conexión del grafo:

```
for nodo in grafo:
```

```
    for vecino in grafo[nodo]:
```

```
        G.add_edge(nodo, vecino)
```

Aquí lo que hice fue recorrer cada nodo del diccionario y luego recorrer sus vecinos para crear las conexiones (aristas) entre ellos dentro del grafo.

Luego definí la posición de los nodos usando posiciones manuales:

```
pos = {
```

```
    "A": (0,0),
```

```
    "B": (-2,-1),
```

```
    "C": (2,-1),
```

```
    "D": (-3,-2),
```

```
    "E": (-1,-2),
```

```
    "F": (1,-2),
```

```
    "G": (3,-2),
```

"H": (-3,-3)

}

Lo hice para organizar el grafo de manera que se vea como un árbol, dejando el nodo A arriba y los demás nodos distribuidos por niveles, las coordenadas (x, y) indican dónde se ubica cada nodo en el plano, los valores negativos o positivos solo sirven para moverlos hacia la izquierda, derecha o hacia abajo, permitiendo que el grafo se vea más claro y facilitando observar los recorridos BFS por niveles y DFS en profundidad.

También implementé los algoritmos DFS, que significa búsqueda en profundidad y BFS que recorre el grafo profundizando lo más posible antes de retroceder.

def dfs(grafo, nodo, visitados): Visita el nodo actual y se guarda en la lista de nodos visitados, luego se revisan todos sus nodos vecinos y para cada uno de ellos, se vuelve a llamar a la función dfs, así el algoritmo sigue avanzando en profundidad dentro del grafo antes de regresar, permitiendo recorrer todos los nodos conectados.

def bfs(grafo, inicio): Utiliza una lista llamada visitados para guardar los nodos que ya se recorrieron y una cola (deque) para controlar el orden y primero se agrega el nodo inicial, luego se va sacando cada nodo, se marca como visitado y se agregan sus vecinos a la cola; este proceso continúa hasta que la cola queda vacía y así, el BFS recorre el grafo por niveles, visitando primero los nodos más cercanos.

Después ejecuté los dos algoritmos comenzando desde el nodo A:

```
visitados_bfs = bfs(grafo, "A")
```

```
visitados_dfs = dfs(grafo, "A", [])
```

Luego organicé e imprimí los resultados en pantalla para ver el orden en que se recorren los nodos y cuántos nodos fueron visitados de manera organizada para que se detalle bien el recorrido de los nodos.

def animar_grafo(G, pos, visitados, titulo): Realiza la animación del recorrido del grafo, la idea es mostrar paso a paso cómo se van visitando los nodos durante la ejecución del algoritmo, lo que permite identificar fácilmente cómo avanza el recorrido progresivo, Utilice la FuncAnimation, que permite actualizar el grafo en cada momento de la animación.

Y finalmente utilice estas dos funciones que generan las dos animaciones, del recorrido BFS y DFS.

```
display(animar_grafo(G, pos, visitados_bfs, "Recorrido BFS"))
```

```
display(animar_grafo(G, pos, visitados_dfs, "Recorrido DFS"))
```